

This is one of the **most reusable interview patterns**. The key insight is:

The stack stores the current best answer. Whenever the current element makes the answer better, remove ("pop") worse previous choices if it's still safe to do so.

Unlike the classic monotonic stack (which looks for next/previous greater/smaller), **Stack + Greedy builds the optimal final sequence**.

When should you think of this pattern?

Look for words like:

- Remove K characters/digits
- Smallest/Largest number
- Lexicographically smallest
- Keep only unique characters
- Maximum number after deletions
- Build the best subsequence

These almost always hint at **Greedy + Stack**.

Core Idea

At each element:

```
Can the current element improve my answer?  
  
YES  
  ↓  
Remove previous worse elements  
  ↓  
Push current element
```

This becomes:

```
while (!st.empty() && top is worse than current && safe_to_remove)  
    st.pop();  
  
st.push(current);
```

The only thing that changes from problem to problem is **what "worse" means** and **what "safe to remove" means**.

General Template

```
for (element : array) {  
  
    while (!st.empty()  
        && current is better than st.top()  
        && removing top is allowed) {  
  
        st.pop();  
    }  
  
    st.push(element);  
}
```

Almost every greedy-stack problem follows this structure.

402. Remove K Digits

Solved 

Medium

 Topics

 Companies

Given string `num` representing a non-negative integer, and an integer `k`, return *the smallest possible integer after removing `k` digits from `num`*.

the intuition is having smaller numbers in the start of the "num"

use a min stack , having minimum numbers at the bottom , when `current < st.top` , you pop

edge cases : `k >= n` , leading zeroes

Here's the same solution with interview-quality comments that explain the **why** instead of just the **what**.

```

// TC = O(n)
// SC = O(n)

string removeKdigits(string num, int k) {
    stack<char> st;
    int n = num.size();

    // Maintain a monotonically increasing stack.
    // If the current digit is smaller than the stack top,
    // removing the larger digit gives a smaller overall number.
    for (int i = 0; i < n; i++) {

        // Greedily remove previous larger digits while we still can.
        while (!st.empty() && k > 0 && st.top() > num[i]) {
            st.pop();
            k--;
        }

        st.push(num[i]);
    }

    // If removals are still left, remove digits from the end
    // since the remaining number is already in increasing order.
    while (k > 0) {
        st.pop();
        k--;
    }

    // All digits removed.
    if (st.empty()) return "0";

    // Stack stores digits in reverse order,
    // so build the string and reverse it.
    string ans = "";
    while (!st.empty()) {
        ans.push_back(st.top());
        st.pop();
    }

    reverse(ans.begin(), ans.end());

    // Remove leading zeros.
    int index = 0;
    while (index < ans.size() && ans[index] == '0')
        index++;

    // Number becomes empty after removing leading zeros.
    if (index == ans.size())
        return "0";

    return ans.substr(index);
}

```

Key idea (worth remembering for interviews)

- **Monotonic Increasing Stack**
- **Greedy choice:** Whenever a smaller digit appears, remove previous larger digits because they contribute more to the number's value.
- If deletions remain after processing all digits, remove from the **end**, since those are the largest remaining positions to sacrifice.
- Finally, strip **leading zeros** and return **"0"** if nothing remains.

316. Remove Duplicate Letters

Solved 

Medium

 Topics

 Companies

 Hint

Given a string `s`, remove duplicate letters so that every letter appears once and only once. You must make sure your result is **the smallest in lexicographical order** among all possible results.

Your solution is already optimal ($O(n)$ time, $O(26)$ space $\approx O(1)$). The only thing I'd improve is readability and remove one redundant check.

Improved Version (Interview Ready)

```
class Solution {
public:
    // TC = O(n)
    // SC = O(1)    // Only 26 lowercase letters

    string removeDuplicateLetters(string s) {
        int n = s.size();

        // last[i] = last occurrence of character ('a' + i)
        vector<int> last(26, -1);

        // inStack[i] = whether character is already present in the stack
        vector<bool> inStack(26, false);

        stack<char> st;

        // Store the last occurrence of every character.
        for (int i = 0; i < n; i++)
            last[s[i] - 'a'] = i;

        // Build the lexicographically smallest subsequence.
        for (int i = 0; i < n; i++) {

            // Skip if this character is already included.
            if (inStack[s[i] - 'a'])
                continue;

            // Remove larger characters if:
            // 1. Current character is smaller.
            // 2. The larger character appears again later.
            while (!st.empty() &&
                    st.top() > s[i] &&
                    last[st.top() - 'a'] > i) {

                inStack[st.top() - 'a'] = false;
                st.pop();
            }

            st.push(s[i]);
            inStack[s[i] - 'a'] = true;
        }

        // Stack stores answer in reverse order.
        string ans;
        while (!st.empty()) {
            ans.push_back(st.top());
            st.pop();
        }

        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

Interview intuition for the `while` loop

```
while (!st.empty() &&  
      st.top() > s[i] &&  
      last[st.top() - 'a'] > i)
```

- `st.top() > s[i]` → Current character makes the answer lexicographically smaller.
- `last[st.top()-'a'] > i` → Safe to remove because this character will appear again later.
- Otherwise, keep it because removing it would lose that character forever.

This is the key greedy idea interviewers usually want you to explain.